

BINARY CLASSIFICATION

ML – Project 1

ABSTRACT: *Binary classification is a pivotal task in the field of machine learning, focusing on categorizing data points into one of two distinct classes. This project delves into various methodologies and algorithms employed for binary classification, including logistic regression, decision trees, K-nearest neighbors and Perceptron, and model evaluation metrics like precision, recall, and F1-score.*

1-INTRODUCTION:

Binary classification is a key aspect of machine learning, allowing systems to make choices based on differentiating between two separate groups. The uses of binary classification, such as identifying spam in emails or making medical diagnoses, have a wide-ranging impact on various fields. This paper offers a thorough summary of the key ideas, methods, and algorithms used in binary classification. It delves into essential methods like logistic regression, decision trees, K-nearest neighbors and Perceptron. Furthermore, the importance of data preprocessing and evaluation metrics in developing strong and dependable classification models is emphasized. This project contains two different types of datasets, one small balanced and the other a large imbalanced.

2-DATASETS:

-Dataset 1: Heart Disease

Heart disease is the leading cause of death in adult human beings. It is important to have predictive models for such a disease as not everyone has medical equipment to get a full diagnosis. This dataset's target is to predict if a person has heart disease or not. It has 12 features columns which includes 1,025 individuals' medical data. The dataset has 7 categorical and 6 quantitative columns.

Encoding of the categorical columns bring the total 26. Some features like chest pain type, max heart rate, and others show a moderate correlation to the target variable. It is fairly balanced dataset. Stratified train test split was used to separate 30% for final testing and 70% for training and cross-validation. Scaling the set didn't have much impact on score (except for Perceptron). This might be due to most of the data already in binary due to encoding categories.

Evaluation Metric: Recall Macro

As it is a riskier medical dataset, my aim is to put emphasis on detecting the disease. The best metric would be recall-macro for this instance. The macro average would evaluate the model's ability to predict both classes. This would result in some healthy individuals being labelled as sick and affect precision and trade-off is calculated by plotting the precision-recall curve.

-Dataset 2: Bank Churn

Customer retention is a major factor for a bank's profitability. This dataset contains details of bank customers and whether they closed their account. Predicting if a customer will be leaving can help the bank take steps to retain them. The dataset originally had 13 feature columns, but certain unique ones like RowNumber, CustomerId and Surname are removed, resulting in 10 total features. Out of these, Geography (3 unique labels) and Gender (2 unique labels) columns were encoded. The final dataset had 13 columns and 10,000 rows. The target class was highly imbalanced, the ratio was almost 4:1 with most people not closing their account. The data was scaled with RobustScaler, as few outliers were noticed in Age and CreditScore columns. Stratified split of 70-30(train-test) was used again, to evenly distribute target values. SMOTE resampling on the training set gave <0.5 scores on the test set so was discarded.

Evaluation Metric: F1-weighted

F1-weighted gave true representation of the models. Initial testing with precision and recall showed model "cheating" and predicting ONLY the class

which leads to higher scores. F1 considers both precision and recall and will help choose hyperparameters to make the classifier learn, instead of predicting a single class

3-ANALYSIS:

Methods: For each dataset and classifier, I have generated validation curves and confusion matrix for most of the hyperparameters. If the validation curve showed better results at a non-default value, the hyperparameter was considered “impactful”. The two most impactful ones are shown in this report. Following that, I used GridSearchCV to find the best combination of them. By doing this, we get the optimal values required for each hyperparameter. Using grid search results, the final learning curve and scores on the test dataset have been generated.

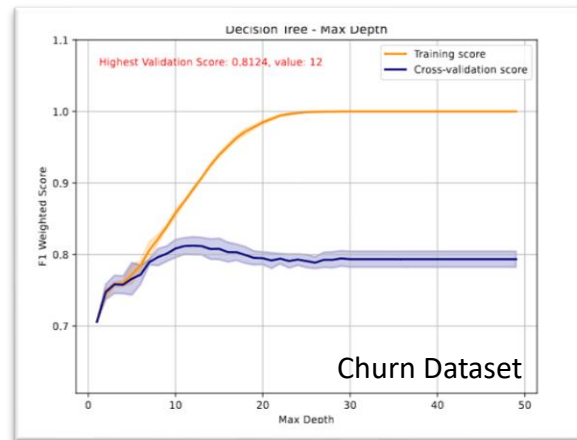
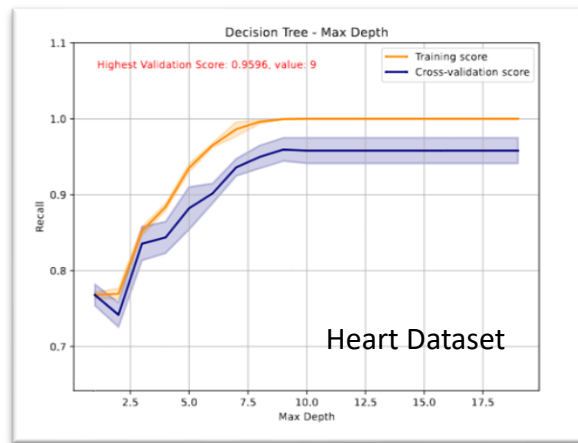
Note: The y-axis of the validation curves is scaled to show minor differences in the score.

I-Decision Tree:

Decision tree does well on both datasets, especially on the heart dataset. Most of the default values were enough to get the highest score on cross-validation. Max-depth was analyzed for both the datasets. The second parameter for heart dataset was max_features; and min_samples_leaf for the churn dataset

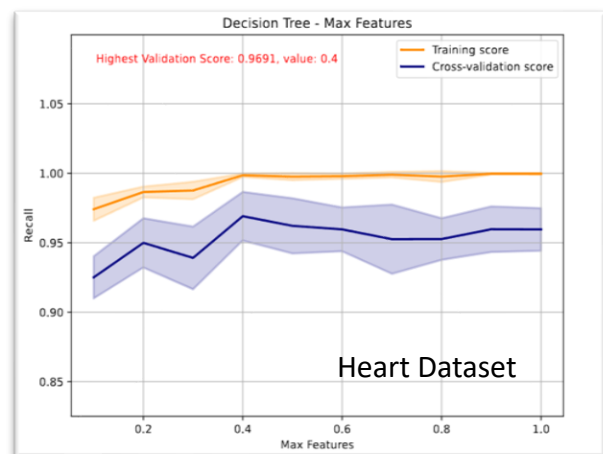
- Max-depth:

Max-depth is the most important here. Tuning it from the default unlimited depth to a smaller number prevented overfitting. It is a parameter that you almost always want to tune to get better generalization. For the heart dataset's recall reaches 0.96 with a max_depth of 9, and then flattens out. The flattening would suggest that the dataset isn't that complex, and the tree doesn't over fit for this data. On the other hand, the churn dataset's optimal depth was at 12 and then the F1 score dropped (from 0.81) before flattening out at 25 max_depth.



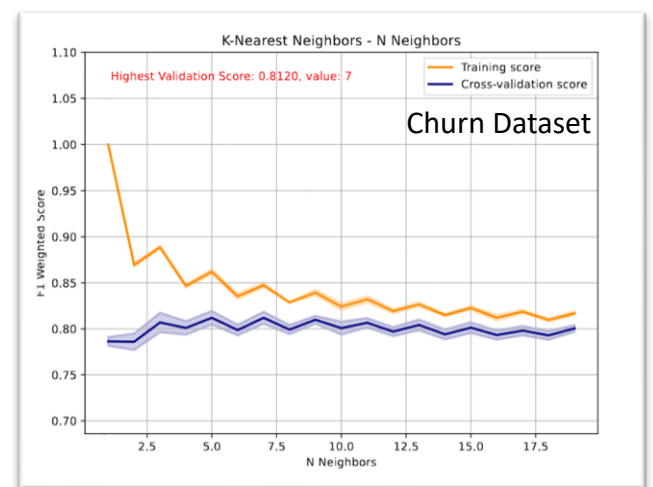
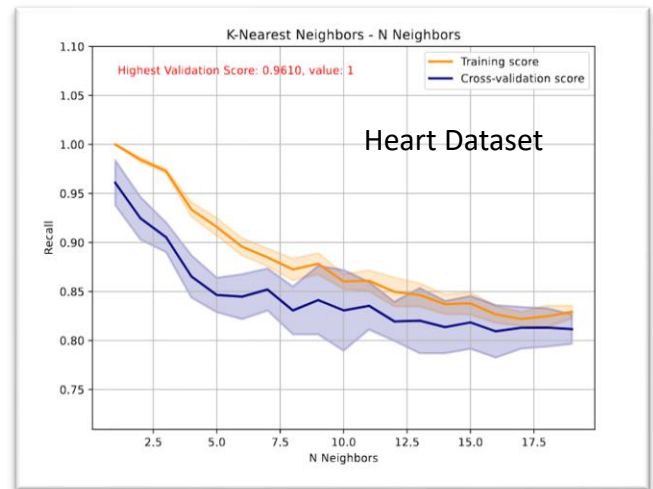
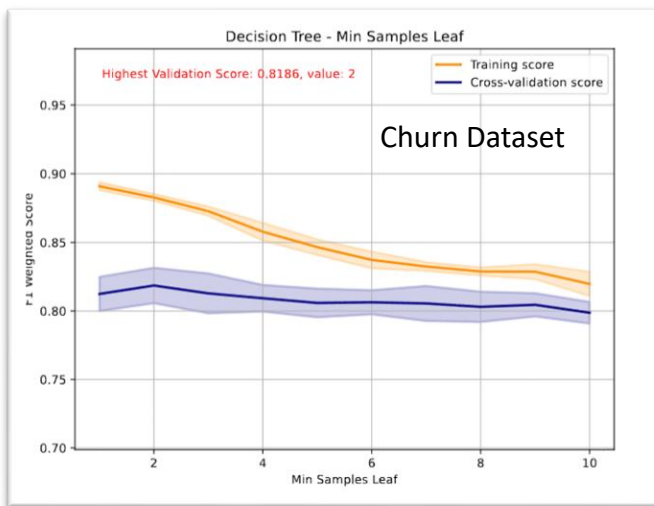
- Max_features

The second parameter for heart dataset was max_features, a fraction representing no. of features considered for the split. It gave the best score at 40% of the total features, again suggesting the dataset isn't complex as it shows a respectable score even when only 40% features are considered at a split.



- Min_sample_leaf

The second parameter for the churn dataset: `min_sample_leaf`, an int that specifies the minimum number of samples a leaf node must have. As shown in the graph the model overfits at the default value of 1 and underfits at 3 and above. The best F1 score is achieved at 2. We have to keep in mind that this dataset is imbalanced so to generalize well, it makes sense to keep `min_sample_leaf` at a higher than default value.



II. K-nearest neighbors (KNN):

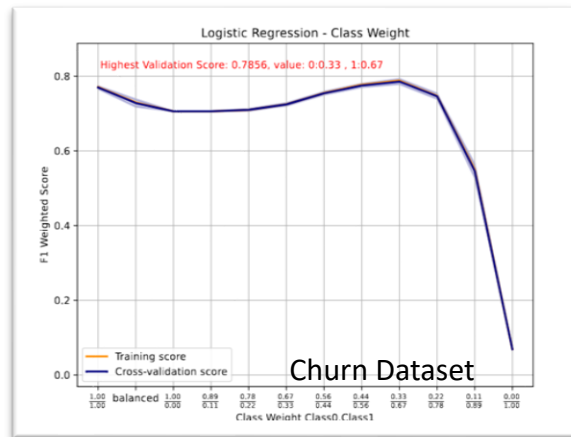
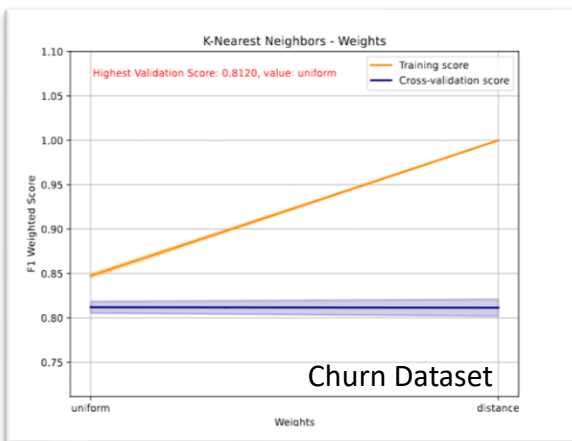
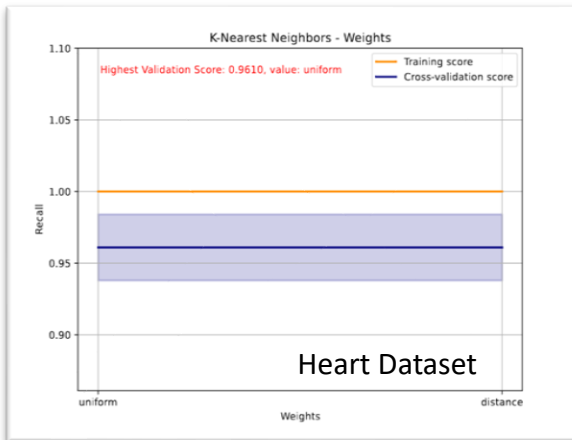
KNN was relatively easy to implement, given that it only has one major hyper parameter: no.of neighbors. The second parameter which I analyzed is the weight function. Other parameter values like p (Minkowski metric power) and algorithm did not have much effect on the scores.

- N_neighbors:

This parameter helps control the precision when predicting, in most cases a value too low will lead to overfitting while also being computationally expensive when predicting the class. The validation curve again proves that the heart dataset is simple as it can achieve the best CV score at a value of 1. The more complex churn data gave best results at a value of 7.

- Weights

This metric is used while prediction of the query. It has two values: uniform, each neighbor is treated equally; and distance, the weight of the neighbor in prediction is proportional to their distance to query. For the heart dataset, it does not matter as the `n_neighbors` value is set to 1. On the other hand, distance function showed more promise on the churn dataset. The CV score was remained same but the training score reached 1 showing room to grow, and combined with other params in GridSearch should give better end result (and it did).



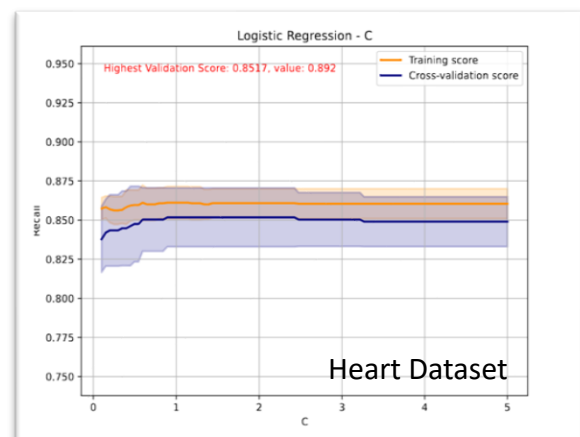
III. Logistic Regression

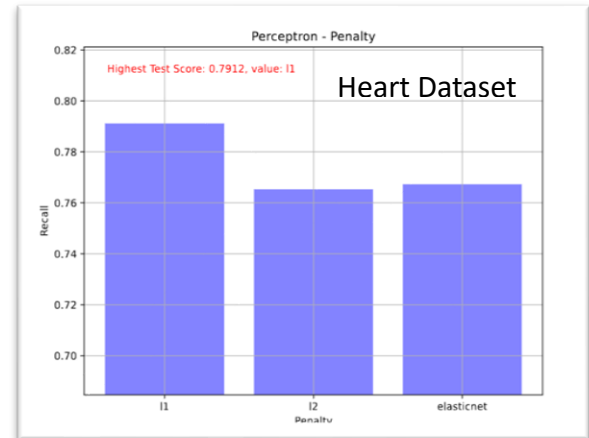
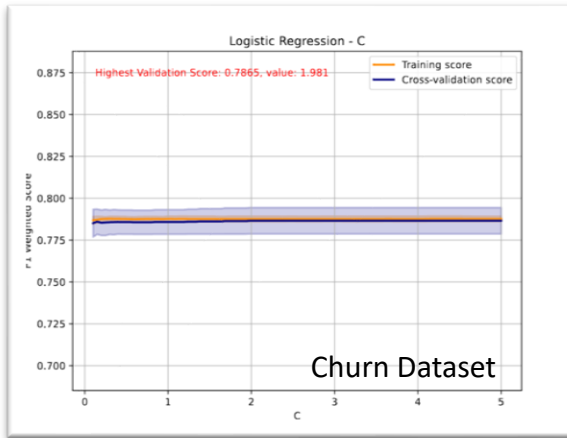
For this classifier, I analyzed the class weight parameters and the regularization strength (C). I was hoping to use elasticnet penalty parameter so the L1_ratio could be bring some improvements but it was negligible. Two things could have happened: the heart data being easy to fit, and the imbalanced churn data was not affected by the penalty. But to cover all bases, I have put the solvers, penalties and l1_ratios in the GridSearch ranges.

Class weight Other than default values of class_weight did not affect the heart dataset validation, except for the extremes. The default value of 1:1 was close enough to the highest CV score with 0.56:0.44. Compared to this the churn data responded well to the class_weight param favoring more weight to class 1, which is the minority class.

• Inverse Regularization Strength (C):

For both datasets, this param didn't have much effect. Using various values of C from 0.1 to 5, there isn't any significant difference between the scores when compared to the default value of 1. Also, I am using L2 penalty for the validation as it is compatible with all the solvers. L2 being less aggressive, maybe doesn't make much impact on the datasets training. From these graphs, I can say none of the features are bring discarded during training and validating.



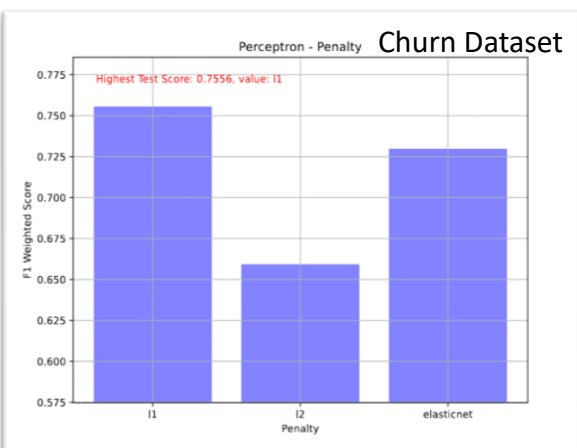


IV. Perceptron

For perceptron, by default it has no penalty initialized, but I wanted to see the difference for different regularization values. So, I compared the penalty and the regularization parameters. Eta0, aka learning is also an important hyperparameter but, there was no difference seen between the default and other values.

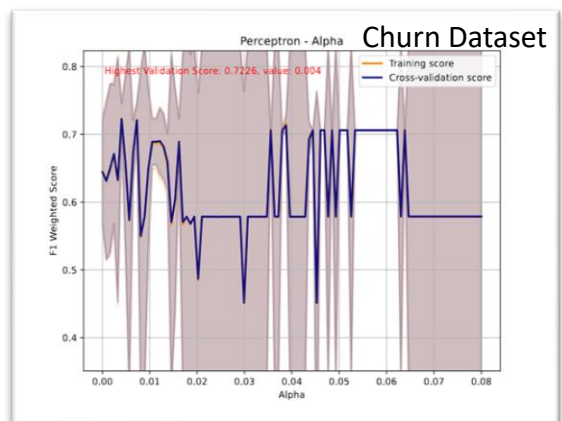
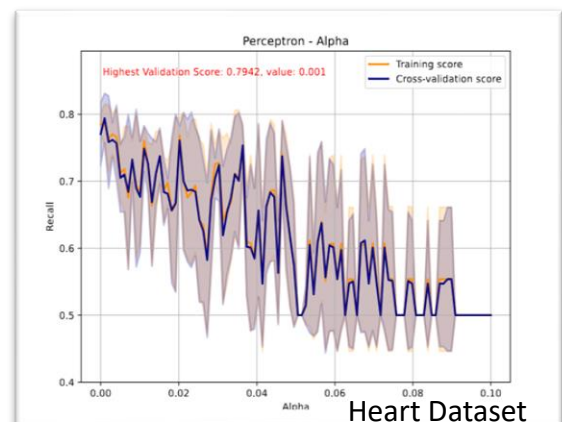
- Penalty:

For both datasets, L1 looked to be performing the best, but we have to keep in mind that this might be misleading considering that the churn dataset is imbalanced. So the aggressive penalty might lead to overfitting in that case.



- Alpha:

This is the regularization term, which determines how heavy the penalty is. The heart dataset achieves the highest score with lowest regularization. For churn dataset the validation bounces between a range, but the best score was at 0.004. In the plot for the churn dataset there are values where there is not any deviation from the mean, it is that these values where the confusion matrix showed it overfitting and predicting just one class for any input.

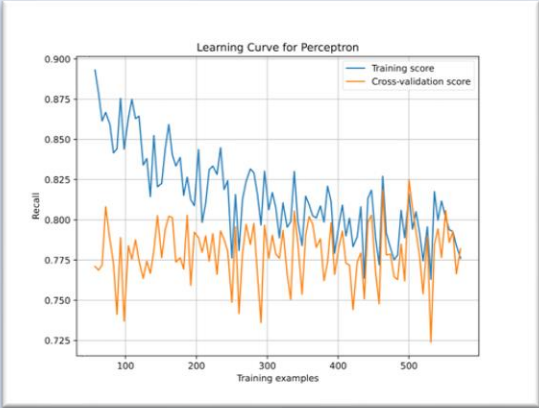
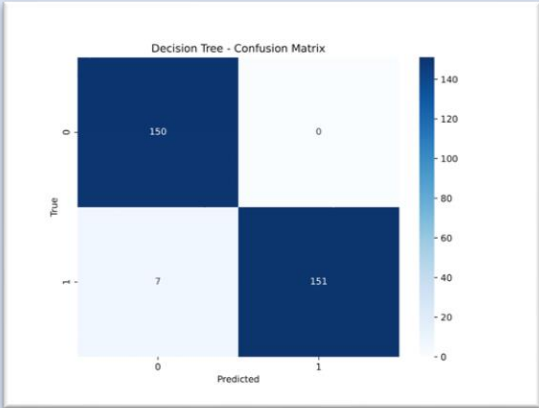
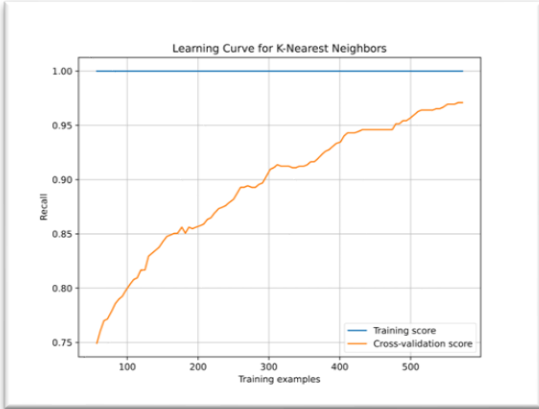
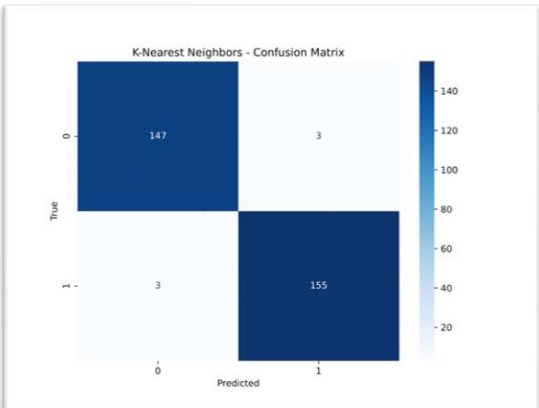


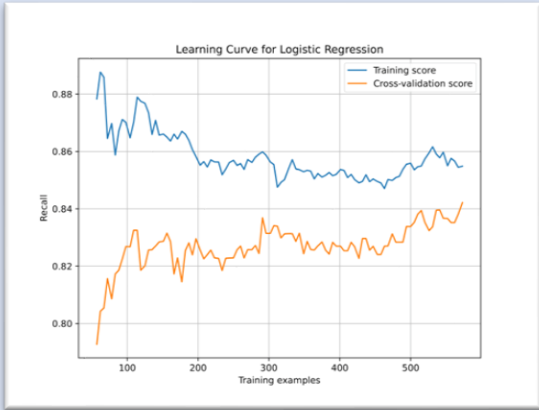
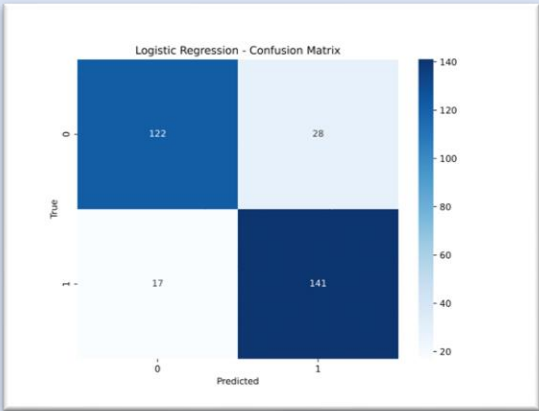
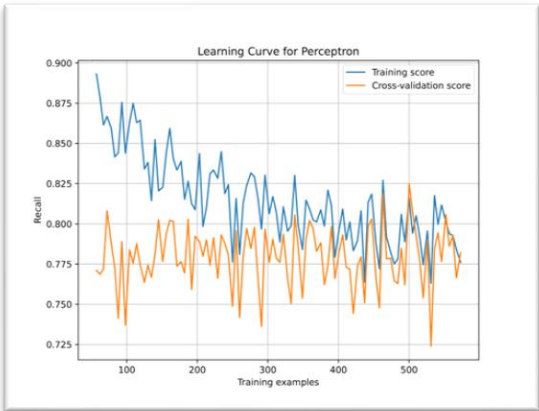
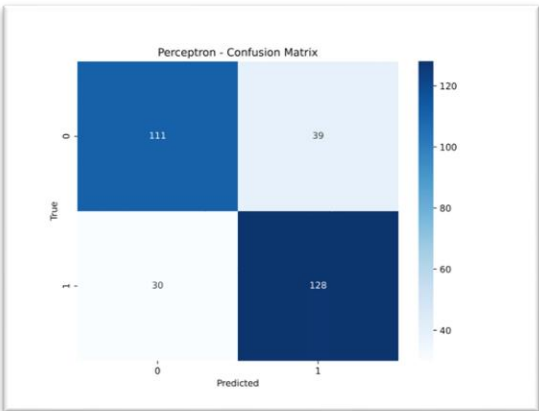
5 - RESULTS (on untouched test set)

Based on the GridSearch results the model was trained and tested on unseen data, its parameters, time to train and score are given along with its learning curves and confusion matrix.

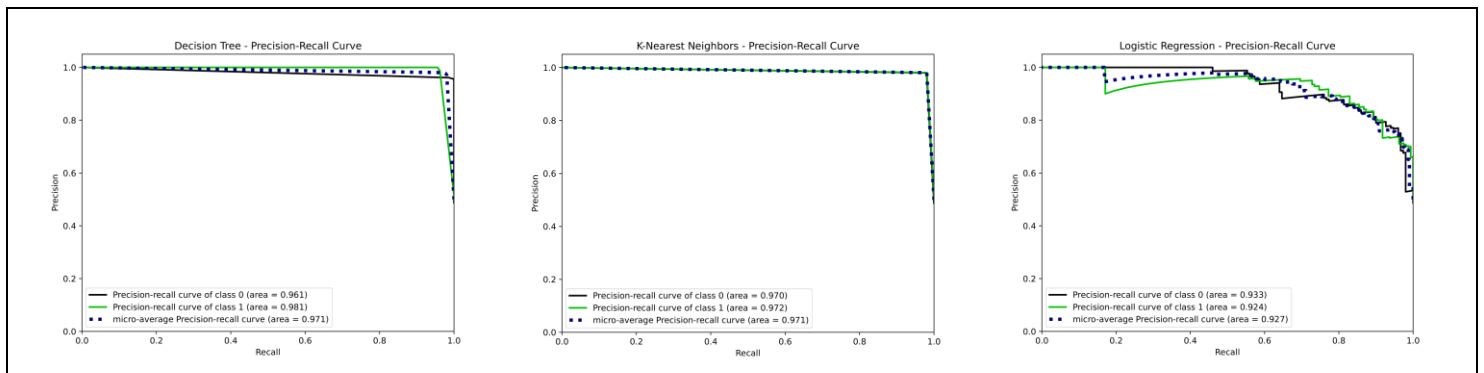
Note: the time to train is the mean of learning_curve fit times.

Dataset 1- Heart Disease

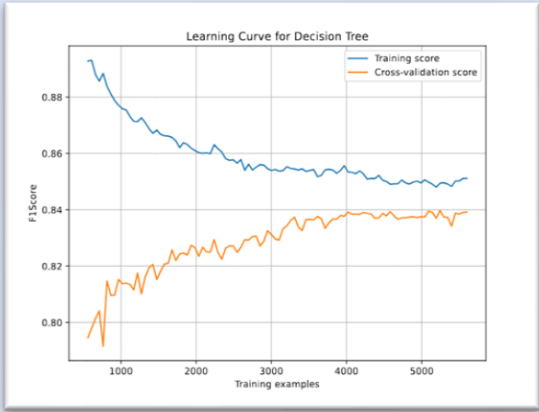
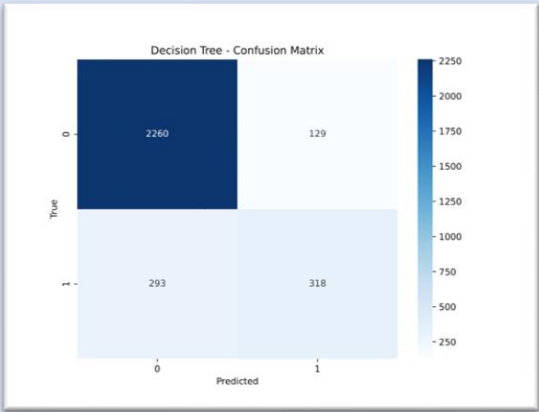
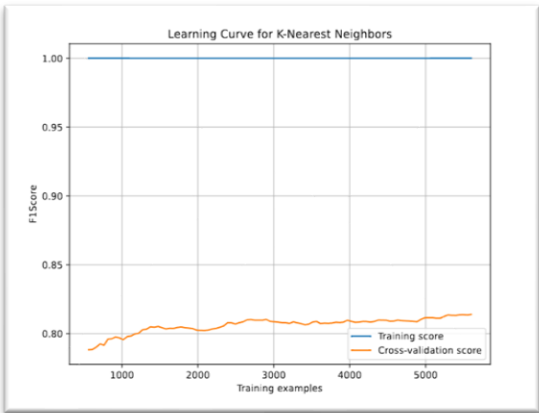
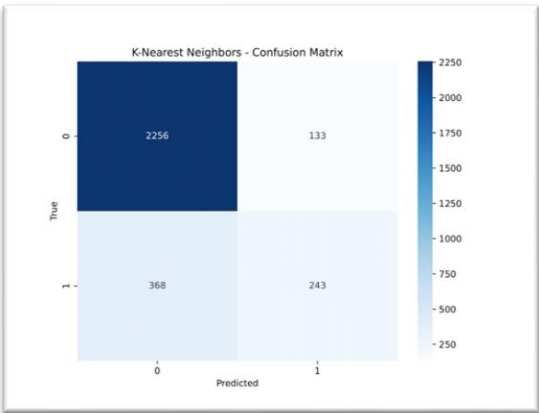
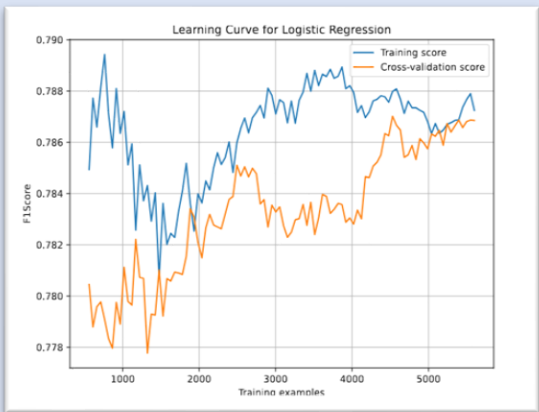
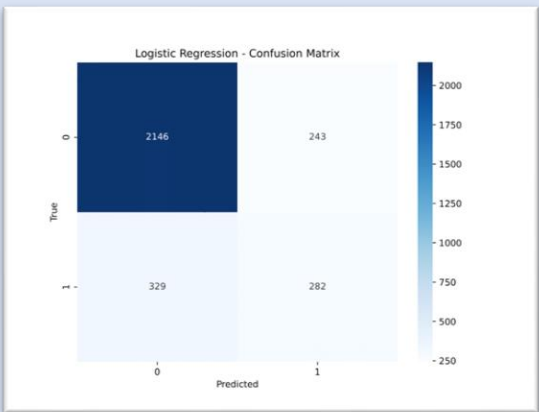
HEART	Parameters (w/ Hyper)	Time to Train	Recall-macro	
			train	test
Decision Tree	 'max_depth=9', 'max_features=0.6'	0.001s	0.999	 0.978
KNN	 'n_neighbors=1', 'p=1'	0.001s	1.0	 0.981

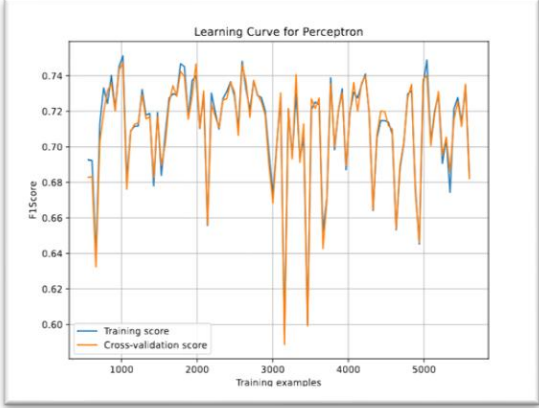
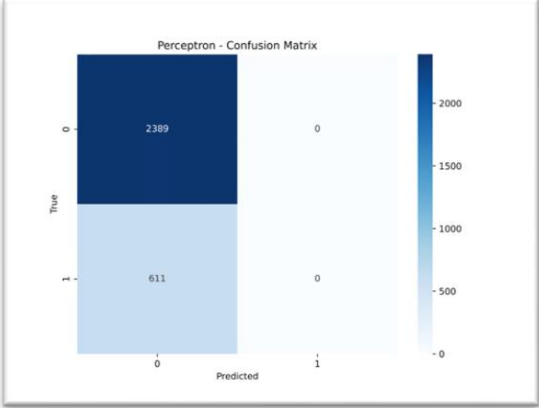
HEART	Parameters (w/ Hyper)	Time to Train	Recall-macro	
			train	test
Logistic Regression	 'C=0.25', 'class_weight={0: 0.44, 1: 0.56}', 'l1_ratio=0.3', 'solver=saga', 'penalty=elasticnet'	0.017s	0.858	 0.853
Perceptron	 'class_weight={0: 0.44, 1: 0.56}', 'l1_ratio=0.2222', 'penalty=elasticnet'	0.001s	0.815	 0.775

Precision-Recall Curves for heart dataset



Dataset 2- Churn Dataset

CHURN	Parameters (w/ Hyper)	Time to Train	F 1-Weighted	
			train	test
Decision Tree	 'class_weight={0: 0.44, 1: 0.56}', 'max_depth=6', 'min_samples_leaf=3', 'min_samples_split=2'	0.008s	0.859	 0.851
KNN	 'n_neighbors=6', 'p=3', 'weights=distance'	0.006s	1.0	 0.817
Logistic Regression	 'C=1.67111', 'class_weight={0: 0.33, 1: 0.67}', 'penalty=l1', 'solver=liblinear'	0.009s	0.786	 0.804

CHURN	Parameters (w/ Hyper)	Time to Train	F 1-Weighted	
			train	test
Perceptron	 'alpha=1e-05', 'class_weight={0: 0.56, 1: 0.44}', 'eta0=0.67333', 'penalty=l1'	0.011s	0.711	 0.706

6 – Discussion

The heart dataset being a simple and comparatively small one gives great results with Decision tree and KNN. Logistic regression and Perceptron give comparable scores while training, but once unseen data is introduced, Perceptron falls way behind. The non-linear models performed significantly better than linear model. This would indicate that a linear relationship is not as prevalent in the dataset. The precision-recall curve shows that the non-linear models are a far better choice than the linear ones. It reaches 95% recall score with negligible amount of precision lost. Also after encoding the dataset, the large feature space helped KNN in order to precisely predict the disease.

The churn dataset was relatively hard to learn for the models. It still gives satisfactory scores, but as the confusion matrix shows the Perceptron has only learned to predict the majority class. Its learning curve is all over the place on the other hand the logistic regression's ability to adjust the probabilities helped it learn the linear relationship in the dataset. Even for the other models, they are not that precise, the score boost is probably from majority class prediction as showing in the confusion matrix. I was disappointed that the class_weight attribute did not

make much of a difference to the linear models. After some research I found out that cleaning data and reducing noise should be a better starting point for an imbalanced dataset.

7 – Conclusion

In conclusion, for the first dataset, I could have chosen a more complicated dataset. A dataset I was considering was the 'Rain in Australia' dataset. It is a fairly large and challenging dataset. Also, in hindsight I could have tried SMOTE resampling with other than its default value or under sample the majority class.

I thoroughly enjoy while working on this project. I came across image classification and neural networks and will be trying to dive into them soon.

8 – References

- [Scikit-learn: Machine Learning in Python](#), Pedregosa *et al.*, JMLR 12, pp. 2825-2830, 2011.
- [Evaluation metrics for imbalanced data](#)
- [Churn Dataset community code- Kaggle](#)
- [Heart Dataset community code -Kaggle](#)